RoMo-GS: Efficient Online 3D Gaussian Splatting for Robot Mapping via Exposure-Integrated Motion Blur

Sangwoo Jung[1,2], Di Deng[1], Jiahui Fu[1], Laura Herlant[1*], Ayoung Kim[2*], Yafei Hu[1*]

[1] RAI Institute, United States [2] Seoul National University, South Korea * Co-corresponding authors

Project Page

arXiv

Personal website

Robotics and AI Institute

Motivation & Overview

Motion blur in robot mapping

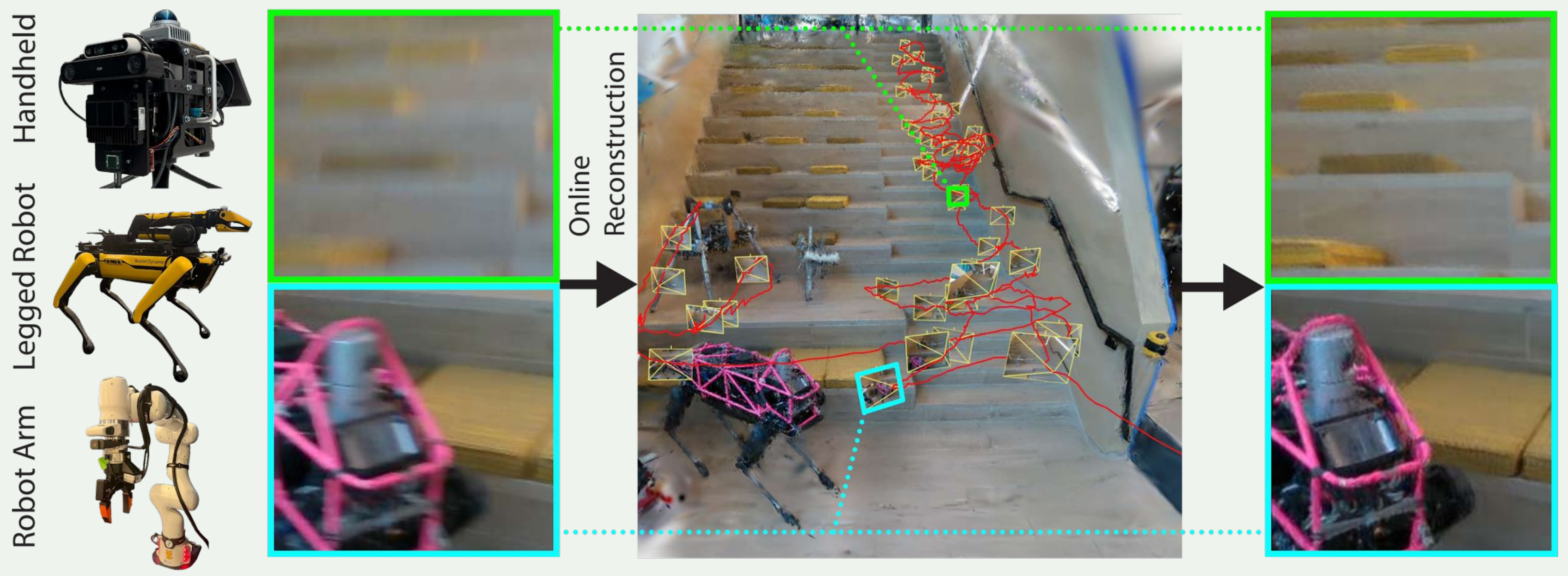
- Blurred observations degrade reconstruction during robot motion.

Computational cost of multi-sampling

- *N* rasterizations per observation, with **O(*N*)** computational cost.
- Larger *N* reduces artifacts but increases latency for online mapping.

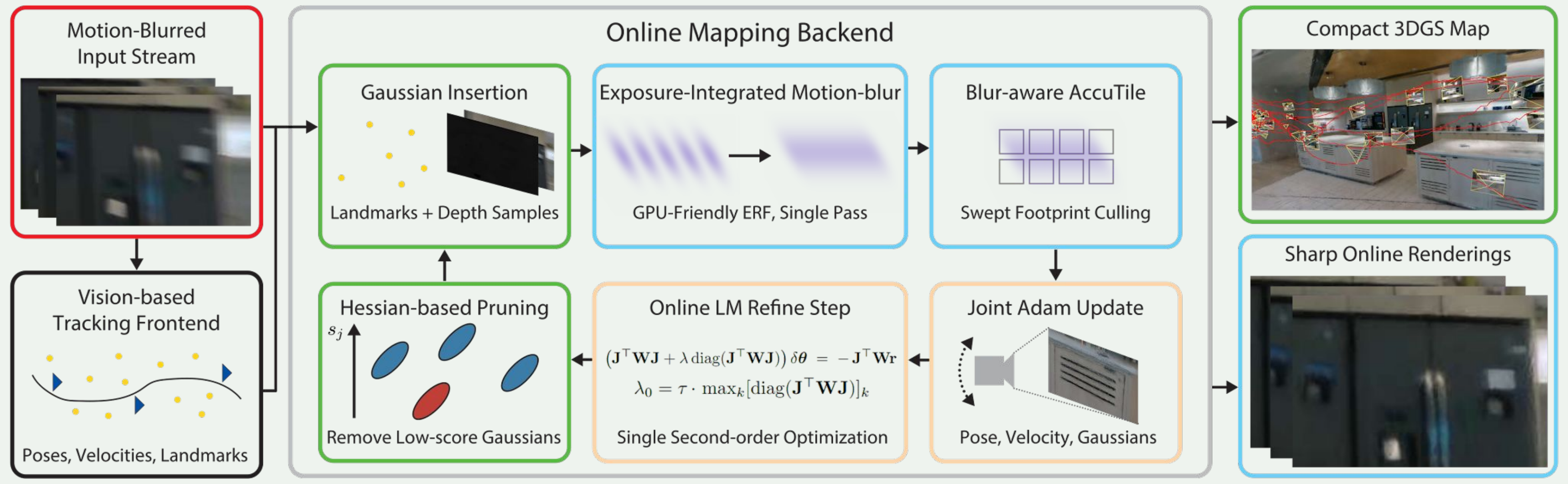
Online map growth

- Redundant Gaussians accumulate during continuous mapping.


Handheld
Legged Robot
Robot Arm
Online Reconstruction

(a) Input Blurry Images from Robots (b) Reconstructed Scene with Camera Trajectory (c) Rendered Sharp Images

Online Mapping Pipeline


Motion-Blurred Input Stream
Vision-based Tracking Frontend
Poses, Velocities, Landmarks
Online Mapping Backend
Gaussian Insertion
Landmarks + Depth Samples
Exposure-Integrated Motion-blur
GPU-Friendly ERF, Single Pass
Blur-aware AccuTile
Swept Footprint Culling
Hessian-based Pruning
s_j
Remove Low-score Gaussians
Online LM Refine Step
$(\mathbf{J}^\top\mathbf{W}\mathbf{J} + \lambda\,\mathrm{diag}(\mathbf{J}^\top\mathbf{W}\mathbf{J}))\,\delta\boldsymbol{\theta} = -\mathbf{J}^\top\mathbf{W}\mathbf{r}$
$\lambda_0 = \tau \cdot \max_k[\mathrm{diag}(\mathbf{J}^\top\mathbf{W}\mathbf{J})]_k$
Single Second-order Optimization
Joint Adam Update
Pose, Velocity, Gaussians
Compact 3DGS Map
Sharp Online Renderings

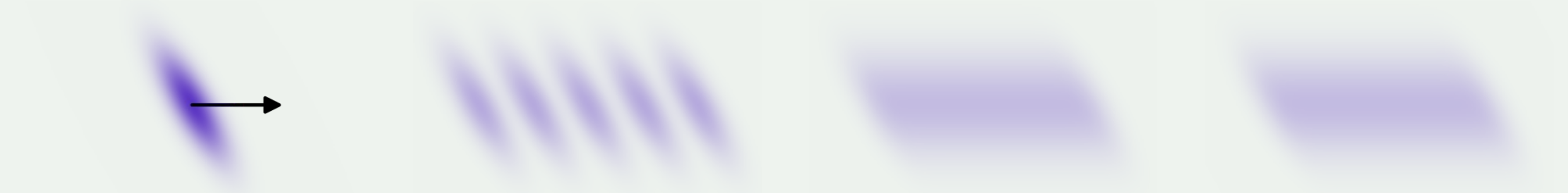
Exposure-Integrated Motion Blur

Screen-space motion

- Integrate projected opacity along a linear screen-space trajectory during the exposure.

$$\overline{\alpha}_j(x) = \frac{1}{t_e}\int_{-t_e/2}^{t_e/2} \alpha_j(x,\ \mu'_j + v'_j\tau)\,\mathrm{d}\tau$$

$\tau \in [-t_e/2,\ t_e/2]$ t_e: exposure duration



(a) Static Gaussian (b) $N_{\text{blur}} = 5$ (c) $N_{\text{blur}} = 100$ (d) Ours ($N_{\text{blur}} = 1$)

Analytic evaluation

- Evaluate opacity with erf in one rasterization pass.
- **O(1)** cost per Gaussian, independent of sample count.

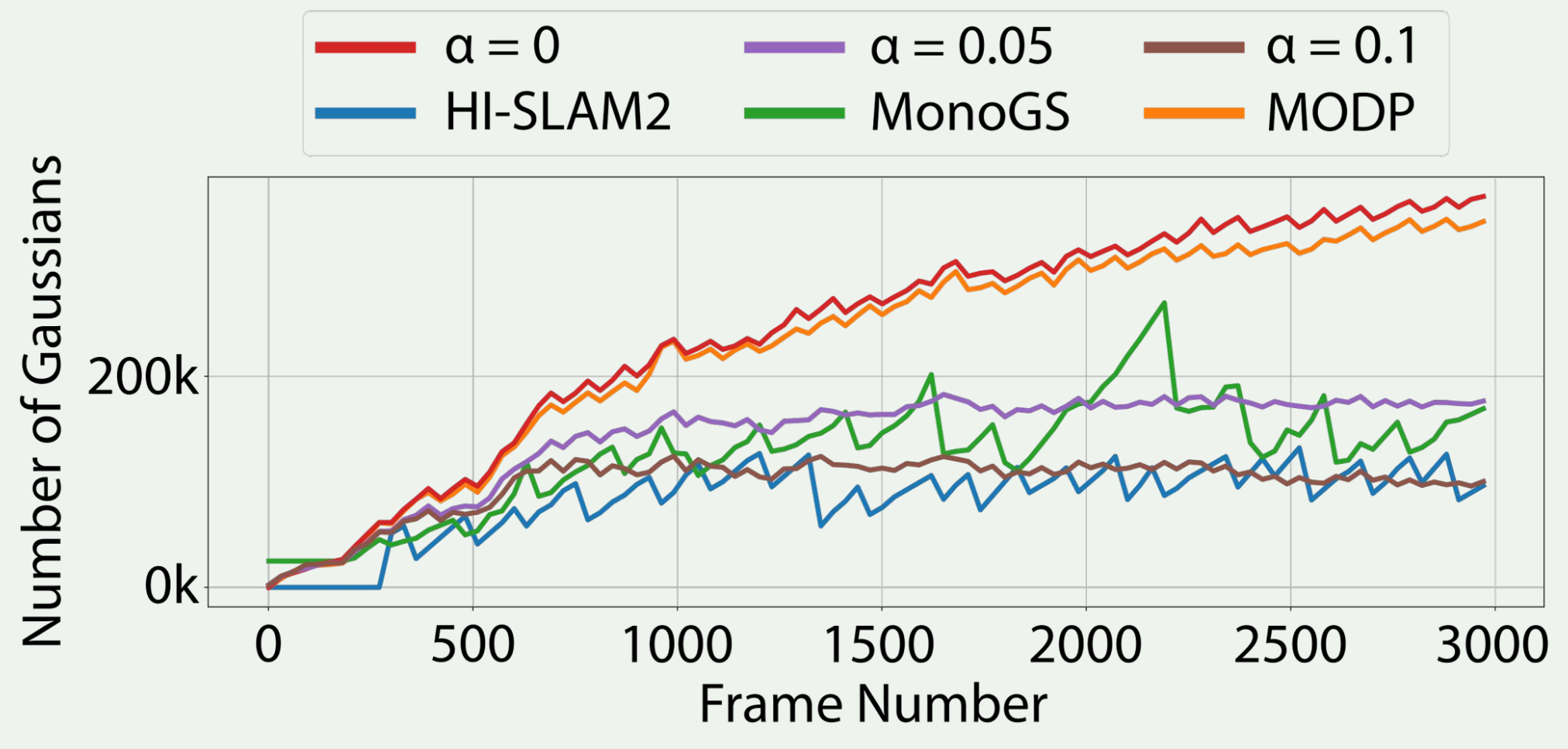
Hessian-Based Online Pruning

Opacity sensitivity

- Reuse opacity gradients from the online photometric loss.

$$s_j = \sum_i \sum_x \left(\frac{\partial L_i}{\partial o_j}\right)^2$$

- Prune low-score Gaussians observed in >10 keyframes.



dog_park: 3,000 frames, 4 loops

~20% fewer Gaussians

compared with MODP

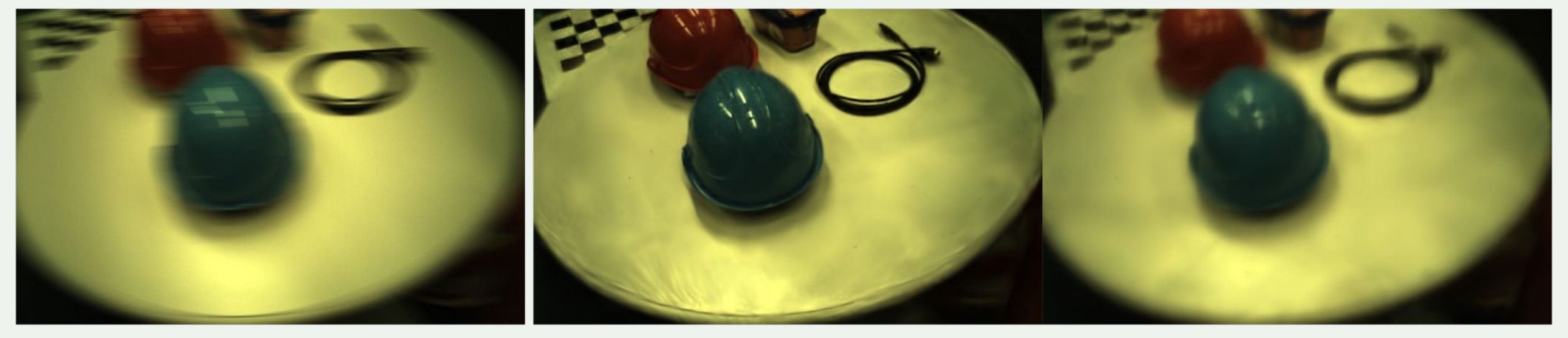
TUM: 167k vs. 210k

Custom: 309k vs. 386k

Comparable quality on TUM and custom datasets (blur model disabled).

Deblurring Quality & Efficiency

Online deblurring on ETH3D

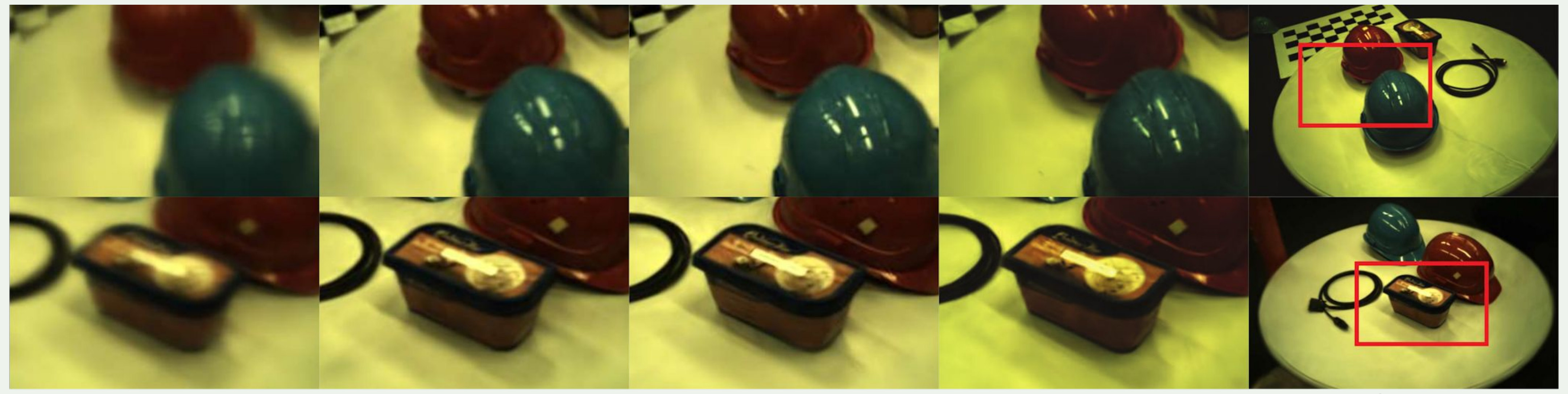


(a) Input Blurry Image (b) Ours (c) MODP

Method	PSNR ↑	LPIPS ↓	# Gaussians
MODP	23.726	0.244	129,165
RoMo-GS	**25.648**	**0.113**	**106,285**

ETH3D camera_shake_1 with BAD-Gaussians reference renderings.

Exposure integration on ETH3D



w/o Blur Reduction $N_{\text{blur}} = 5$ $N_{\text{blur}} = 10$ Ours Reference

Method	PSNR ↑	SSIM ↑	LPIPS ↓	FPS ↑	# Gaussians ↓
w/o blur reduction	24.355	0.892	0.202	5.698	129,128
Multi-sample, *N* = 5	25.951	0.911	0.121	2.448	124,831
Multi-sample, *N* = 10	26.003	0.911	0.107	1.592	126,902
RoMo-GS	25.367	0.905	0.113	3.695	117,669

2.32× faster than *N* = 10 multi-sampling

ETH3D comparison using the same mapping pipeline and BAD-Gaussians reference renderings.

Deblur-NeRF synthetic

~15–149× faster than offline baselines (total optimization time)

Deblurring 3DGS GSOM BAGS CoMoGaussian Ours Reference